ELECTRICITY BILL MANAGEMENT SYSTEM

DBMS PROJECT REPORT

CONTENTS

1. INTRODUCTION 3

1.1 Overview 3

1.2 Problem Statement 3

1.3 Objectives 3

1.4 Scope & Limitations 4

2. SYSTEM FUNCTIONAL REQUIREMENTS 5

2.1 Functional Requirements 5

2.2 Hardware & Software Requirements 6

3. DATABASE DESIGN 7

3.1 Conceptual Design (E-R Diagram) 7

3.2 Logical Design (Relational Schema Diagram) 8

3.3 Normalization 9

4. IMPLEMENTATION 14

4.1 Database Creation (DDL Statements) 14

4.2 DML Statements 15

4.3 SQL Queries 16

4.4 Database Connectivity 19

4.5 Data Retrieval Code 20

4.6 Functions Used 21

4.7 Triggers 24

4.8 Sample Page 25

5. SYSTEM TESTING 27

5.1 Test Cases 27

5.2 Screenshots of Work 30

6. CONCLUSION 34

7. FUTURE SCOPE 35

8. APPENDIX 36

9. REFERENCES 37

1. INTRODUCTION

1.1 Overview

Electricity is a basic need of every household, shop and factory, and every consumer is billed periodically for the energy used. When billing is done manually or in spreadsheets, the amount is worked out by hand from the meter reading, which leads to calculation errors, duplicate bills and records that are hard to trace. The Electricity Bill Management System (VoltCare) is a database-driven web application that automates this work. It keeps consumer, connection, meter-reading and billing data in one centralized MySQL database and calculates every bill automatically using a slab-wise (progressive) tariff: the first block of units is charged at the lowest rate and every further block at a higher rate.

The system has three layers. The data layer is a MySQL database of six related tables designed from the ER diagram. The application layer is a Python Flask REST API that connects to MySQL, validates every request and performs the bill calculation inside a database transaction. The presentation layer is a browser dashboard written in HTML, CSS and JavaScript from which the user registers consumers and connections, records meter readings, generates bills, views the slab-wise breakdown and updates the payment status. The slab pattern follows the style used by electricity boards such as KSEB; the rates used in this project are illustrative values chosen for academic demonstration and are not the official KSEB tariff.

1.2 Problem Statement

Developing an Electricity Bill Database Management System that ensures accurate bill generation and provides a transparent slab-wise breakdown of charges, enabling consumers to clearly understand the cost of each unit of electricity consumed.

In a manual process the consumer sees only a final amount and cannot tell how it was obtained, errors in slab arithmetic go unnoticed, and the same reading can be billed twice. The proposed system removes these problems by calculating the bill in one place, storing every slab line that makes up the amount, and allowing only one bill for each meter reading.

1.3 Objectives

- To ensure accurate bill calculation and eliminate errors in the amount generated.
- To provide a transparent, slab-wise itemized breakdown of charges on every bill.
- To maintain a reliable, structured database of consumer, connection and billing records for easy retrieval and auditing.
- To enforce data integrity using primary keys, foreign keys, unique and check constraints.
- To provide a simple web interface for registering consumers, recording meter readings, generating bills and tracking payment status.
- To produce a printable bill that can be saved as a PDF.

1.4 Scope & Limitations

Scope: The system covers consumer registration (with one or more addresses), connection management, periodic meter-reading entry, automatic slab-wise bill generation, storage of the slab breakdown, payment-status tracking, a summary dashboard and a printable bill. All data is stored in MySQL and retrieved through a REST API, so any other client (for example a mobile application) can reuse the same back end.

Limitations:

- The tariff in the application is an illustrative academic tariff and the same slab table is applied to Domestic, Commercial and Industrial connections.
- The current version has no login or role-based access and no online-payment gateway; payment status is updated manually.
- The consumed units are entered by the user for each billing period (they are not calculated from previous and current meter readings).
- The planned Flagged_Status (misuse detection) from the design is not yet implemented in the application.
- Consumers, connections and readings are added through the web page; editing and deleting them is done with SQL statements (Section 4.3).

2. SYSTEM FUNCTIONAL REQUIREMENTS

2.1 Functional Requirements

1. Consumer Management

- The system shall register a consumer with Consumer_ID (auto-generated), Name, Phone and Email; the Email must be unique.
- The system shall store one or more addresses for each consumer (multivalued attribute) in a separate table.
- The system shall list all consumers together with their addresses.

2. Connection Management

- The system shall allow a consumer to own one or more connections, each with a unique Meter_No.
- Each connection shall have a type (Domestic, Commercial or Industrial) and a status (Active or Inactive).

3. Meter Reading (Usage) Management

- The system shall record a usage record with Read_Date and Units_Consumed for a connection.
- The system shall reject negative units and shall not accept readings for inactive connections.

4. Bill Generation

- The system shall generate exactly one bill for each usage record, with Bill_Date, Due_Date and an optional Subsidy.
- The system shall calculate the energy charge on the server using progressive slabs; the user cannot type the final amount.
- Final_Amount = Energy_Charge − Subsidy, and it shall never be negative.

5. Slab Breakdown

- The system shall store one SLAB_BREAKDOWN row (slab range, units, rate, amount) for every slab used by a bill, inserted in the same transaction as the bill.

6. Payment Management

- Each bill shall have a Payment_Status (Pending, Paid or Overdue); the user can mark a pending bill as Paid.

7. Bill Presentation

- The system shall display a printable bill showing the consumer, meter, units, slab-wise charges, subsidy and final amount, and allow it to be saved as PDF.

8. Dashboard and Reporting

- The dashboard shall show total consumers, active connections, bills generated and pending bills, and list the consumers, connections, usage records, bills and slab breakdowns.

9. Data Integrity and Validation

- The system shall validate every request, return clear error messages, and keep the data consistent through constraints and transactions (all-or-nothing bill creation).

2.2 Hardware & Software Requirements

Hardware Requirements

Table 2.1: Hardware requirements

Component	Server / Development machine	Client machine
Processor	Intel Core i3 or equivalent (i5 recommended)	Any dual-core processor
RAM	4 GB minimum, 8 GB recommended	2 GB or higher
Storage	5 GB free space (MySQL, Python and project files)	Not significant (browser only)
Display	1366 × 768 or higher	1280 × 720 or higher
Network	Localhost / LAN (no internet needed to run)	LAN or broadband access to the server

Software Requirements

Table 2.2: Software requirements

Category	Software used
Operating system	Windows 10/11, Linux or macOS
Database	MySQL Server 8.0 (MySQL Workbench / command line to run the SQL script)
Back-end language	Python 3.10 or above (tested with Python 3.13)
Web framework	Flask 3.x (REST API)
Libraries	Flask-CORS 4.x–5.x, mysql-connector-python 8.3 or above (listed in requirements.txt)
Front end	HTML5, CSS3, JavaScript (ES6, Fetch API)
Tools	Visual Studio Code with Live Server, draw.io (ER diagram), a modern web browser (Chrome / Edge / Firefox)

3. DATABASE DESIGN

3.1 Conceptual Design (E-R Diagram)

The conceptual design is shown in Fig 3.1 (Chen notation). It has five entities: CONSUMER, CONNECTION, USAGE_RECORD, BILL and SLAB_BREAKDOWN. Key attributes are underlined, Address is a multivalued attribute (double oval) and the double lines show total participation.

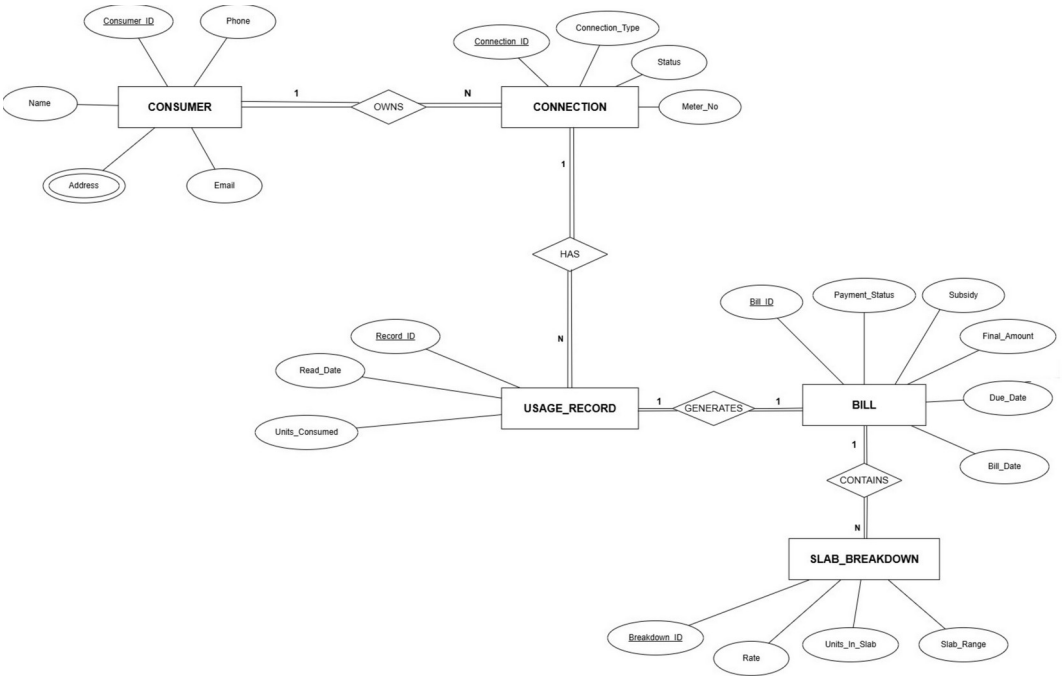


Fig 3.1: ER Diagram of the Electricity Bill Management System

The relationships are OWNS (CONSUMER 1:N CONNECTION), HAS (CONNECTION 1:N USAGE_RECORD), GENERATES (USAGE_RECORD 1:1 BILL) and CONTAINS (BILL 1:N SLAB_BREAKDOWN).

Assumptions

1. Each consumer can have many electricity connections. Hence, the relationship between CONSUMER and CONNECTION is 1:N.
2. Each connection is assigned a unique Connection_ID and Meter_No. No two connections share the same meter.
3. One connection can have multiple usage records. A new usage record is created for every billing cycle (e.g., every month).
4. Each usage record generates exactly one bill. Therefore, the relationship between USAGE_RECORD and BILL is 1:1.
5. Each bill may contain multiple slab breakdown entries, and different unit ranges are charged at different rates.
6. Every bill belongs to only one usage record.
7. Consumer details (Name, Phone, Address, Email) are stored only once to avoid data duplication.

8. Payment_Status stores whether the bill is Paid, Unpaid (Pending) or Overdue.
9. Connection Status indicates whether the connection is Active or Inactive.
10. Flagged_Status is a derived attribute calculated from electricity usage (for example, unusually high consumption) and is not entered manually.
11. All Primary Keys are unique.
12. Bills are generated only after a valid usage record is recorded for the corresponding connection.

3.2 Logical Design (Relational Schema Diagram)

The ER diagram was converted to relations with the standard ER-to-relational mapping algorithm. Each entity became a relation; the multivalued attribute Address became its own relation, CONSUMER_ADDRESS, with the composite key (Consumer_ID, Address); for every 1:N relationship the foreign key was placed on the 'many' side; and for the 1:1 relationship GENERATES the foreign key Record_ID was placed in BILL and made UNIQUE. The resulting relational schema is shown in Fig 3.2; the arrows point from each foreign key to the primary key it references.

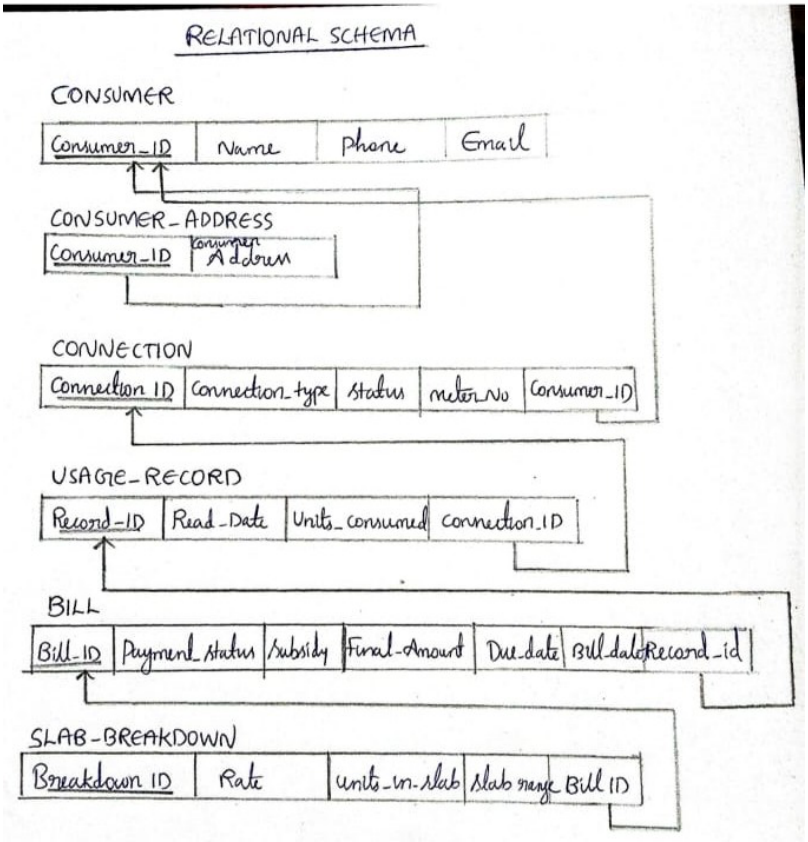


Fig 3.2: Relational Schema of the Electricity Bill Management System

Design versus implementation. While implementing the schema in MySQL, a few names and columns were adjusted. They are listed in Table 3.1 so that the report matches the code exactly.

Table 3.1: Differences between the design and the MySQL implementation

Design (Fig 3.2)	Implemented in MySQL	Reason
CONNECTION	CONNECTION_DETAILS	CONNECTION is also an SQL keyword, so a descriptive table name is used
CONSUMER_ADDRESS. Address	CONSUMER_ADDRESS. Consumer_Address	A clearer column name
—	BILL.Energy_Charge	Stores the slab total before subsidy so that Final_Amount = Energy_Charge − Subsidy can be audited
—	SLAB_BREAKDOWN. Slab_Amount	Stores Units_In_Slab × Rate for every slab line
BILL.Payment_Status (Paid, Unpaid)	BILL.Payment_Status ENUM('Pending', 'Paid', 'Overdue')	Pending is the unpaid state; Overdue is allowed for bills past the due date

3.3 Normalization

Normalization is the process of organizing the tables of a database so that redundancy is removed and insert, update and delete anomalies are avoided. The relational schema of Fig 3.2 was checked step by step against 1NF, 2NF, 3NF and BCNF. Each step below states the rule, applies it to the tables of this project and gives the conclusion.

Unnormalized starting point

If the whole system were kept in a single sheet, one row would have to carry the consumer, every address of that consumer, the connection, the meter reading, the bill and all of its slab lines. For example, the bill of Anu Krishnan would repeat her name, phone and e-mail for every bill and every slab line, and a consumer with two addresses would need a repeating group. Such a table suffers from update anomalies (changing a phone number means changing many rows), insertion anomalies (a consumer cannot be stored before a bill exists) and deletion anomalies (deleting the only bill removes the consumer). Normalization removes these problems by decomposing the sheet into the six relations shown in Fig 3.2.

1) First Normal Form (1NF)

Rule: Every attribute must hold only atomic (indivisible) values; multivalued attributes and repeating groups are not allowed.

In our schema:

- The entity CONSUMER has a multivalued attribute Address (double oval in the ER diagram), because a consumer may have more than one address.
- To make the attribute atomic it is moved to a new relation CONSUMER_ADDRESS (Consumer_ID, Consumer_Address) whose primary key is the combination of both

columns.

- The slab lines of a bill (a repeating group) are stored as separate rows of SLAB_BREAKDOWN instead of repeated columns inside BILL.
- Every remaining attribute (Name, Phone, Email, Connection_Type, Status, Meter_No, Read_Date, Units_Consumed, Payment_Status, Subsidy, Final_Amount, Due_Date, Bill_Date, Rate, Units_In_Slab, Slab_Range) stores a single value.

Conclusion: All tables satisfy 1NF.

2) Second Normal Form (2NF)

Rule: The relation must be in 1NF and there must be no partial dependency, i.e. every non-key attribute must depend on the whole primary key and not on only a part of a composite key.

In our schema:

- CONSUMER, CONNECTION, USAGE_RECORD, BILL and SLAB_BREAKDOWN have single-attribute primary keys (Consumer_ID, Connection_ID, Record_ID, Bill_ID, Breakdown_ID), so a partial dependency is not possible.
- CONSUMER_ADDRESS has the composite key (Consumer_ID, Consumer_Address) and no other attribute, so there is nothing that could depend on only a part of the key.

Conclusion: All tables satisfy 2NF.

3) Third Normal Form (3NF)

Rule: The relation must be in 2NF and there must be no transitive dependency, i.e. a non-key attribute must not depend on another non-key attribute.

In our schema:

- CONSUMER: Name, Phone and Email depend only on Consumer_ID.
- CONNECTION: Connection_Type, Status, Meter_No and Consumer_ID depend only on Connection_ID. The owner's details are not repeated here; they are reached through the foreign key Consumer_ID.
- USAGE_RECORD: Read_Date, Units_Consumed and Connection_ID depend only on Record_ID. Meter and consumer details are not repeated.
- BILL: Payment_Status, Subsidy, Final_Amount, Due_Date, Bill_Date and Record_ID depend only on Bill_ID.
- SLAB_BREAKDOWN: Rate, Units_In_Slab, Slab_Range and Bill_ID depend only on Breakdown_ID.
- No table has a non-key attribute that determines another non-key attribute.

Conclusion: All tables satisfy 3NF.

4) Boyce-Codd Normal Form (BCNF)

Rule: The relation must be in 3NF and for every functional dependency $X \rightarrow Y$, X must be a super key.

Table 3.2: Functional dependencies and candidate keys

Relation	Functional dependencies	Candidate keys
CONSUMER	Consumer_ID → Name, Phone, Email; Email → Consumer_ID	Consumer_ID, Email
CONSUMER_ADDRESS	Only the trivial dependency on (Consumer_ID, Consumer_Address)	(Consumer_ID, Consumer_Address)
CONNECTION	Connection_ID → all attributes; Meter_No → Connection_ID	Connection_ID, Meter_No
USAGE_RECORD	Record_ID → Read_Date, Units_Consumed, Connection_ID	Record_ID
BILL	Bill_ID → all attributes; Record_ID → Bill_ID	Bill_ID, Record_ID
SLAB_BREAKDOWN	Breakdown_ID → Rate, Units_In_Slab, Slab_Range, Bill_ID	Breakdown_ID

- CONNECTION: Meter_No is unique to each connection (no two connections share a meter), so it is a candidate key and the dependency Meter_No → Connection_ID is valid.
- BILL: GENERATES is a strict 1:1 relationship, so the foreign key Record_ID is unique in BILL and works as a candidate key.
- CONSUMER: Email is declared UNIQUE, so Email → Consumer_ID is also a dependency on a candidate key.
- In every relation each determinant is a primary key or a candidate key, hence a super key.

Conclusion: All tables satisfy BCNF. The schema of Fig 3.2 is already normalized and needs no further decomposition.

Final decomposition

Table 3.3: Normalized relations of the project

Relation	Primary key	Foreign key
CONSUMER (Consumer_ID, Name, Phone, Email)	Consumer_ID	—
CONSUMER_ADDRESS (Consumer_ID, Consumer_Address)	(Consumer_ID, Consumer_Address)	Consumer_ID → CONSUMER
CONNECTION (Connection_ID, Connection_Type, Status, Meter_No, Consumer_ID)	Connection_ID	Consumer_ID → CONSUMER
USAGE_RECORD (Record_ID, Read_Date, Units_Consumed, Connection_ID)	Record_ID	Connection_ID → CONNECTION
BILL (Bill_ID, Payment_Status, Subsidy, Final_Amount, Due_Date, Bill_Date, Record_ID)	Bill_ID	Record_ID → USAGE_RECORD (UNIQUE)
SLAB_BREAKDOWN (Breakdown_ID, Rate, Units_In_Slab, Slab_Range, Bill_ID)	Breakdown_ID	Bill_ID → BILL

Note on stored computed values. In the MySQL implementation the columns Slab_Amount (= Units_In_Slab × Rate), Energy_Charge (= sum of the slab amounts) and Final_Amount (= Energy_Charge − Subsidy) can be derived from other columns. They are stored deliberately as a snapshot of the bill: if the tariff changes later, old bills must still show the amount that was actually charged. Only the back end writes these values, inside one transaction, and the query in Section 4.3 checks that the slab amounts always add up to the Energy_Charge. This is a controlled exception to strict normalization and is the only place where a stored value is derivable from other stored values.

Normalization Summary

The table below summarizes the result of each normal-form check for every relation of the project.

Table 3.4: Normal forms satisfied by each relation

Relation	1NF	2NF	3NF	BCNF
CONSUMER	Yes	Yes	Yes	Yes
CONSUMER_ADDRESS	Yes	Yes	Yes	Yes
CONNECTION	Yes	Yes	Yes	Yes
USAGE_RECORD	Yes	Yes	Yes	Yes
BILL	Yes	Yes	Yes	Yes
SLAB_BREAKDOWN	Yes	Yes	Yes	Yes

Normalized Relational Schema

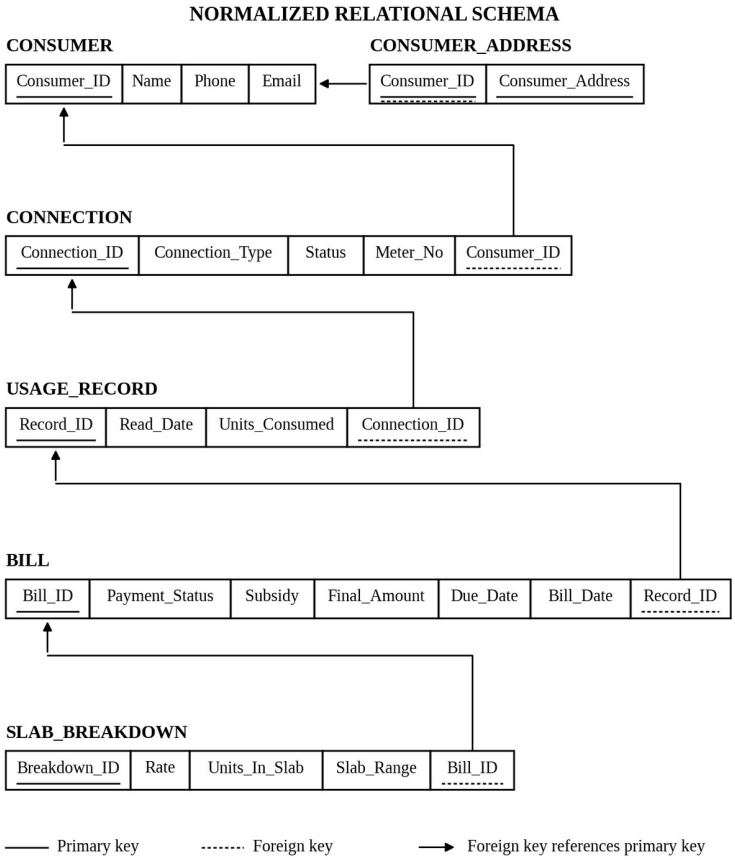


Fig 3.3: Normalized relational schema (PK solid underline, FK dashed underline, arrows from FK to PK)

4. IMPLEMENTATION

4.1 Database Creation (DDL Statements)

The script schema.sql creates the database electricity_bill_db and the six tables. Foreign keys use ON DELETE CASCADE only for addresses (they have no meaning without the consumer) and ON DELETE RESTRICT elsewhere, so billing history cannot be deleted by accident. ENUM, UNIQUE, NOT NULL and CHECK constraints keep invalid data out of the tables.

```
CREATE DATABASE IF NOT EXISTS electricity_bill_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE electricity_bill_db;
-- For a clean first-time installation, run this file in a new project database.

CREATE TABLE IF NOT EXISTS CONSUMER (
  Consumer_ID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Phone INT(10) NOT NULL CHECK (Phone REGEXP '^[0-9]{10}$'),
  Email VARCHAR(120) NOT NULL,
  UNIQUE KEY uq_consumer_email (Email)
);

CREATE TABLE IF NOT EXISTS CONSUMER_ADDRESS (
  Consumer_ID INT NOT NULL,
  Consumer_Address VARCHAR(255) NOT NULL,
  PRIMARY KEY (Consumer_ID, Consumer_Address),
  CONSTRAINT fk_address_consumer FOREIGN KEY (Consumer_ID)
    REFERENCES CONSUMER(Consumer_ID) ON DELETE CASCADE ON UPDATE
      CASCADE
);

CREATE TABLE IF NOT EXISTS CONNECTION_DETAILS (
  Connection_ID INT AUTO_INCREMENT PRIMARY KEY,
  Connection_Type ENUM('Domestic','Commercial','Industrial') NOT NULL,
  Status ENUM('Active','Inactive') NOT NULL DEFAULT 'Active',
  Meter_No VARCHAR(50) NOT NULL UNIQUE,
  Consumer_ID INT NOT NULL,
  CONSTRAINT fk_connection_consumer FOREIGN KEY (Consumer_ID)
    REFERENCES CONSUMER(Consumer_ID) ON DELETE RESTRICT ON UPDATE
      CASCADE
);

CREATE TABLE IF NOT EXISTS USAGE_RECORD (
  Record_ID INT AUTO_INCREMENT PRIMARY KEY,
  Read_Date DATE NOT NULL,
  Units_Consumed DECIMAL(10,2) NOT NULL,
  Connection_ID INT NOT NULL,
  CONSTRAINT chk_units_nonnegative CHECK (Units_Consumed >= 0),
```

```
  CONSTRAINT fk_usage_connection FOREIGN KEY (Connection_ID)
    REFERENCES CONNECTION_DETAILS(Connection_ID) ON DELETE
       RESTRICT ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS BILL (
  Bill_ID INT AUTO_INCREMENT PRIMARY KEY,
  Payment_Status ENUM('Pending','Paid','Overdue') NOT NULL DEFAULT 'Pending',
  Subsidy DECIMAL(10,2) NOT NULL DEFAULT 0,
  Energy_Charge DECIMAL(10,2) NOT NULL,
  Final_Amount DECIMAL(10,2) NOT NULL,
  Due_Date DATE NOT NULL,
  Bill_Date DATE NOT NULL,
  Record_ID INT NOT NULL UNIQUE,
  CONSTRAINT chk_subsidy_nonnegative CHECK (Subsidy >= 0),
  CONSTRAINT chk_energy_nonnegative CHECK (Energy_Charge >= 0),
  CONSTRAINT chk_amount_nonnegative CHECK (Final_Amount >= 0),
  CONSTRAINT fk_bill_usage FOREIGN KEY (Record_ID)
    REFERENCES USAGE_RECORD(Record_ID) ON DELETE RESTRICT ON
       UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS SLAB_BREAKDOWN (
  Breakdown_ID INT AUTO_INCREMENT PRIMARY KEY,
  Rate DECIMAL(10,2) NOT NULL,
  Units_In_Slab DECIMAL(10,2) NOT NULL,
  Slab_Range VARCHAR(50) NOT NULL,
  Slab_Amount DECIMAL(10,2) NOT NULL,
  Bill_ID INT NOT NULL,
  CONSTRAINT chk_rate_nonnegative CHECK (Rate >= 0),
  CONSTRAINT chk_slab_units_nonnegative CHECK (Units_In_Slab >= 0),
  CONSTRAINT chk_slab_amount_nonnegative CHECK (Slab_Amount >= 0),
  CONSTRAINT fk_slab_bill FOREIGN KEY (Bill_ID)
    REFERENCES BILL(Bill_ID) ON DELETE RESTRICT ON UPDATE CASCADE,
  INDEX idx_slab_bill (Bill_ID)
);
```

4.2 DML Statements

The script then inserts three dummy consumers with their addresses, connections, usage records, bills and slab rows. The slab rows of each bill add up to its Energy_Charge, and Final_Amount = Energy_Charge − Subsidy.

```
-- Illustrative dummy data. Do not use these tariff amounts as official KSEB rates.
INSERT INTO CONSUMER (Name,Phone,Email) VALUES
('Anu Krishnan',9876543210,'anu.demo@example.com'),
('Rahul Nair',9876543211,'rahul.demo@example.com'),
('Meera Das',9876543212,'meera.demo@example.com');
```

```
INSERT INTO CONSUMER_ADDRESS (Consumer_ID,Consumer_Address) VALUES
(1,'House 12, Ernakulam, Kerala'),
(2,'Green Villa, Thrissur, Kerala'),
(3,'Rose House, Kottayam, Kerala');

INSERT INTO CONNECTION_DETAILS
   (Connection_Type,Status,Meter_No,Consumer_ID) VALUES
('Domestic','Active','MTR-DEMO-1001',1),
('Domestic','Active','MTR-DEMO-1002',2),
('Commercial','Active','MTR-DEMO-1003',3);

INSERT INTO USAGE_RECORD (Read_Date,Units_Consumed,Connection_ID)
   VALUES
('2026-10-01',120,1),
('2026-10-02',245,2),
('2026-10-03',45,3);

-- Energy charges based on the example progressive slabs:
-- 120 units = (50*3.25)+(50*4.15)+(20*5.25) = Rs.475.00
-- 245 units = (50*3.25)+(50*4.15)+(100*5.25)+(45*6.50) = Rs.1,187.50
-- 45 units = 45*3.25 = Rs.146.25
INSERT INTO BILL
  (Payment_Status,Subsidy,Energy_Charge,Final_Amount,Due_Date,Bill_Date,Record_I
     D)
VALUES
('Pending',20.00,475.00,455.00,'2026-10-25','2026-10-11',1),
('Paid',0.00,1187.50,1187.50,'2026-10-26','2026-10-12',2),
('Pending',10.00,146.25,136.25,'2026-10-27','2026-10-12',3);

INSERT INTO SLAB_BREAKDOWN
   (Rate,Units_In_Slab,Slab_Range,Slab_Amount,Bill_ID) VALUES
(3.25,50,'0-50 units',162.50,1),
(4.15,50,'51-100 units',207.50,1),
(5.25,20,'101-200 units',105.00,1),
(3.25,50,'0-50 units',162.50,2),
(4.15,50,'51-100 units',207.50,2),
(5.25,100,'101-200 units',525.00,2),
(6.50,45,'Above 200 units',292.50,2),
(3.25,45,'0-50 units',146.25,3);
```

4.3 SQL Queries

This section shows CRUD operations, JOIN queries, aggregate functions and sub queries. The outputs shown are those of the demo database after Bills 1 and 2 were marked as paid (compare with Fig 5.7).

CRUD operations

```
-- CREATE
INSERT INTO CONSUMER (Name, Phone, Email)
  VALUES ('Arun Kumar', 9876500002, 'arun.kumar@example.com');
INSERT INTO CONSUMER_ADDRESS (Consumer_ID, Consumer_Address)
  VALUES (LAST_INSERT_ID(), 'Sree Nivas, Palakkad, Kerala');

-- READ
SELECT * FROM CONSUMER;
SELECT * FROM CONSUMER WHERE Name = 'Arun Kumar';

-- UPDATE
UPDATE CONSUMER SET Phone = 9876500099 WHERE Email =
    'arun.kumar@example.com';

-- DELETE (its address row is removed by ON DELETE CASCADE)
DELETE FROM CONSUMER WHERE Email = 'arun.kumar@example.com';

-- The same operations on the other tables
INSERT INTO CONNECTION_DETAILS (Connection_Type, Status, Meter_No,
    Consumer_ID)
  VALUES ('Industrial', 'Active', 'MTR-DEMO-1005', 1);
UPDATE CONNECTION_DETAILS SET Status = 'Inactive' WHERE Meter_No =
    'MTR-DEMO-1005';
INSERT INTO USAGE_RECORD (Read_Date, Units_Consumed, Connection_ID)
  VALUES ('2026-11-01', 180, 1);
UPDATE USAGE_RECORD SET Units_Consumed = 185 WHERE Record_ID = 5;
DELETE FROM USAGE_RECORD WHERE Record_ID = 5;

-- UPDATE on BILL (mark a bill as paid) and READ of a bill
UPDATE BILL SET Payment_Status = 'Paid' WHERE Bill_ID = 1;
SELECT * FROM BILL WHERE Bill_ID = 1;
```

JOIN query – bills with consumer details

```
SELECT b.Bill_ID, c.Name AS Consumer, u.Units_Consumed,
  b.Energy_Charge, b.Subsidy, b.Final_Amount, b.Payment_Status
FROM BILL b
JOIN USAGE_RECORD u ON u.Record_ID = b.Record_ID
JOIN CONNECTION_DETAILS cd ON cd.Connection_ID = u.Connection_ID
JOIN CONSUMER c ON c.Consumer_ID = cd.Consumer_ID
ORDER BY b.Bill_ID;
```

Output:

Bill_ID	Consumer	Units_ Consumed	Energy_ Charge	Subsidy	Final_ Amount	Payment_ Status
1	Anu Krishnan	120.00	475.00	20.00	455.00	Paid

Bill_ID	Consumer	Units_ Consumed	Energy_ Charge	Subsidy	Final_ Amount	Payment_ Status
2	Rahul Nair	245.00	1187.50	0.00	1187.50	Paid
3	Meera Das	45.00	146.25	10.00	136.25	Pending

JOIN query – slab-wise itemised bill of Bill 2

```
SELECT sb.Slab_Range, sb.Units_In_Slab, sb.Rate, sb.Slab_Amount
FROM SLAB_BREAKDOWN sb
JOIN BILL b ON b.Bill_ID = sb.Bill_ID
WHERE b.Bill_ID = 2
ORDER BY sb.Breakdown_ID;
```

Output:

Slab_Range	Units_In_Slab	Rate	Slab_Amount
0–50 units	50.00	3.25	162.50
51–100 units	50.00	4.15	207.50
101–200 units	100.00	5.25	525.00
Above 200 units	45.00	6.50	292.50

Aggregate functions – COUNT and SUM per consumer

```
SELECT c.Name, COUNT(b.Bill_ID) AS Total_Bills,
  SUM(u.Units_Consumed) AS Total_Units,
  SUM(b.Final_Amount) AS Total_Billed
FROM CONSUMER c
JOIN CONNECTION_DETAILS cd ON cd.Consumer_ID = c.Consumer_ID
JOIN USAGE_RECORD u ON u.Connection_ID = cd.Connection_ID
JOIN BILL b ON b.Record_ID = u.Record_ID
GROUP BY c.Consumer_ID, c.Name;
```

Output:

Name	Total_Bills	Total_Units	Total_Billed
Anu Krishnan	1	120.00	455.00
Rahul Nair	1	245.00	1187.50
Meera Das	1	45.00	136.25

Sub query – consumers who have an unpaid bill

```
SELECT Consumer_ID, Name FROM CONSUMER
WHERE Consumer_ID IN (
  SELECT cd.Consumer_ID
  FROM CONNECTION_DETAILS cd
  JOIN USAGE_RECORD u ON u.Connection_ID = cd.Connection_ID
  JOIN BILL b ON b.Record_ID = u.Record_ID
  WHERE b.Payment_Status <> 'Paid');
```

Output:

Consumer_ID	Name
3	Meera Das

Integrity check – slab amounts must add up to the energy charge

```
SELECT b.Bill_ID, b.Energy_Charge, SUM(sb.Slab_Amount) AS Slab_Total
FROM BILL b JOIN SLAB_BREAKDOWN sb ON sb.Bill_ID = b.Bill_ID
GROUP BY b.Bill_ID, b.Energy_Charge
HAVING b.Energy_Charge <> SUM(sb.Slab_Amount);
```

The query returns an empty set on the demo database, which proves that every stored bill is consistent with its slab breakdown.

4.4 Database Connectivity

The Flask application (app.py) connects to MySQL with the mysql-connector-python driver. The connection parameters are read from environment variables, so the database password is never written in the source code or in the JavaScript that runs in the browser. A new connection is opened for each request and closed in a finally block.

```
def get_db():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST", "localhost"),
        user=os.getenv("DB_USER", "root"),
        password=os.getenv("DB_PASSWORD", ""),
        database=os.getenv("DB_NAME", "electricity_bill_db"),
        port=int(os.getenv("DB_PORT", "3306")),
    )

app = Flask(__name__)
CORS(app)  # lets the browser page call the API from another port
```

The variables are DB_HOST (default localhost), DB_PORT (3306), DB_USER (root), DB_PASSWORD (empty) and DB_NAME (electricity_bill_db). On the browser side, script.js talks to the API through one small wrapper around the Fetch API. Every call sends and receives JSON, and any error message returned by Flask is shown to the user as a notification:

```
const API = "http://127.0.0.1:5000/api";
async function api(path, options = {}) {
  const response = await fetch(API + path, {
    ...options,
    headers: { "Content-Type": "application/json", ...(options.headers || {}) }
  });
  const body = await response.json()
    .catch(() => ({ error: "Invalid response from server" }));
  if (!response.ok)
    throw new Error(body.error || `Request failed (${response.status})`);
  return body;
}
```

4.5 Data Retrieval Code

All data shown on the dashboard is retrieved with parameterised SELECT statements. The helper query_all() opens a connection, runs the query, converts the rows to dictionaries and closes the connection. json_value() converts MySQL DECIMAL values to numbers and DATE values to ISO text so that they can be sent as JSON.

```
def rows_to_dicts(cursor, rows):
    return [
        {col[0]: json_value(value) for col, value in zip(cursor.description, row)}
        for row in rows
    ]

def query_all(sql, params=()):
    db = get_db()
    try:
        cur = db.cursor()
        cur.execute(sql, params)
        return rows_to_dicts(cur, cur.fetchall())
    finally:
        db.close()
```

Retrieving a bill with its slab breakdown. The bill is read with a four-table join (BILL, USAGE_RECORD, CONNECTION_DETAILS, CONSUMER), and its slab rows are read with a second query using the bill number as a parameter (the %s placeholder prevents SQL injection):

```
@app.get("/api/bills/<int:bill_id>")
def get_bill(bill_id):
    rows = query_all(
        """
        SELECT b.Bill_ID, b.Payment_Status, b.Subsidy, b.Energy_Charge,
               b.Final_Amount, b.Due_Date, b.Bill_Date,
               b.Record_ID, u.Units_Consumed, u.Read_Date, cd.Connection_ID,
               cd.Meter_No, cd.Consumer_ID, c.Name AS Consumer_Name
        FROM BILL b JOIN USAGE_RECORD u ON u.Record_ID=b.Record_ID
```

```
        JOIN CONNECTION_DETAILS cd ON cd.Connection_ID=u.Connection_ID
        JOIN CONSUMER c ON c.Consumer_ID=cd.Consumer_ID WHERE b.Bill_ID=%s
        """,
        (bill_id,),
    )
    if not rows:
        return error_response("Bill not found.", 404)
    bill = rows[0]
    bill["slab_breakdown"] = query_all(
        """SELECT Breakdown_ID, Bill_ID, Slab_Range, Units_In_Slab, Rate,
                Slab_Amount
           FROM SLAB_BREAKDOWN WHERE Bill_ID=%s ORDER BY
             Breakdown_ID""",
        (bill_id,),
    )
    return jsonify(bill)
```

Refreshing the page. In script.js, refresh() requests the health check and the five list endpoints together with Promise.all, stores the results in a cache and calls render(), which redraws the tables, the dashboard counters and the drop-down lists.

4.6 Functions Used

Bill calculation – calculate_slabs()

This is the most important function of the project. It receives the units consumed and walks through the tariff list. In each slab it takes as many of the remaining units as the slab can hold, multiplies them by the slab rate and rounds to two decimals; the last slab has no limit. Decimal arithmetic is used instead of floating point so that money is never rounded incorrectly.

```
TARIFF = [
    {"limit": Decimal("50"), "rate": Decimal("3.25"), "label": "0-50 units"},
    {"limit": Decimal("50"), "rate": Decimal("4.15"), "label": "51-100 units"},
    {"limit": Decimal("100"), "rate": Decimal("5.25"), "label": "101-200 units"},
    {"limit": None, "rate": Decimal("6.50"), "label": "Above 200 units"},
]

def calculate_slabs(units):
    remaining = Decimal(str(units)).quantize(CENT, rounding=ROUND_HALF_UP)
    result, total = [], Decimal("0.00")
    lower = Decimal("0")
    for slab in TARIFF:
        if remaining <= 0:
            break
        available = (
            remaining if slab["limit"] is None else min(remaining, slab["limit"])
        )
        amount = (available * slab["rate"]).quantize(CENT, rounding=ROUND_HALF_UP)
        upper = "above" if slab["limit"] is None else str(lower + slab["limit"])
```

```
        result.append({
            "Slab_Range": slab["label"],
            "Units_In_Slab": available,
            "Rate": slab["rate"],
            "Slab_Amount": amount,
        })
        total += amount
        remaining -= available
        if slab["limit"] is not None:
            lower += slab["limit"]
    return result, total.quantize(CENT, rounding=ROUND_HALF_UP)
```

Worked example (245 units). The first 50 units fall in slab 1, the next 50 in slab 2, the next 100 in slab 3 and the remaining 45 units in the last slab:

Slab	Units in slab	Rate (Rs./unit)	Amount (Rs.)
0–50 units	50	3.25	162.50
51–100 units	50	4.15	207.50
101–200 units	100	5.25	525.00
Above 200 units	45	6.50	292.50
Energy charge	**245**		**1187.50**

For 120 units the charge is 50 × 3.25 + 50 × 4.15 + 20 × 5.25 = Rs. 475.00; with a subsidy of Rs. 20.00 the final amount is Rs. 455.00.

Bill generation inside a transaction – add_bill()

A bill and all its slab rows must be created together or not at all. add_bill() therefore starts a transaction, locks the usage record with SELECT ... FOR UPDATE (so two users cannot bill the same reading at the same time), refuses a second bill for the same record, calls calculate_slabs(), inserts the BILL row and one SLAB_BREAKDOWN row per slab, and commits. If anything fails, rollback() undoes everything. The main part of the function is shown below.

```
@app.post("/api/bills")
def add_bill():
    # ... read the JSON body and check Record_ID, Bill_Date, Due_Date ...
    try:
        # ... convert and validate Subsidy ...
        db = get_db()
        try:
            db.start_transaction()
            cur = db.cursor(dictionary=True)
            cur.execute(
                """SELECT u.Record_ID, u.Units_Consumed FROM USAGE_RECORD u
                   WHERE u.Record_ID=%s FOR UPDATE""",
                (d["Record_ID"],),
            )
```

```
        usage = cur.fetchone()
        if not usage:
            db.rollback()
            return error_response("Usage record not found.", 404)
        cur.execute(
            "SELECT Bill_ID FROM BILL WHERE Record_ID=%s", (d["Record_ID"],)
        )
        if cur.fetchone():
            db.rollback()
            return error_response("A bill already exists for this usage record.")
        slab_rows, energy = calculate_slabs(usage["Units_Consumed"])
        final = max(Decimal("0.00"), energy - subsidy).quantize(
            CENT, rounding=ROUND_HALF_UP
        )
        cur.execute(
            """INSERT INTO BILL(Payment_Status, Subsidy, Energy_Charge,
                    Final_Amount, Due_Date, Bill_Date, Record_ID)
               VALUES(%s, %s, %s, %s, %s, %s, %s)""",
            ("Pending", subsidy, energy, final,
             d["Due_Date"], d["Bill_Date"], d["Record_ID"]),
        )
        bid = cur.lastrowid
        for slab in slab_rows:
            cur.execute(
                """INSERT INTO SLAB_BREAKDOWN(Rate, Units_In_Slab, Slab_Range,
                        Slab_Amount, Bill_ID)
                   VALUES(%s, %s, %s, %s, %s)""",
                (slab["Rate"], slab["Units_In_Slab"], slab["Slab_Range"],
                 slab["Slab_Amount"], bid),
            )
        db.commit()
        # ... return JSON with Bill_ID, Energy_Charge, Final_Amount, slab rows ...
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
except (ValueError, TypeError):
    # ... return a clear error message ...
    pass
```

Other back-end and front-end functions

Table 4.1: Back-end functions

Function (Python / Flask)	Purpose
get_db()	Opens a MySQL connection using the environment variables
query_all(sql, params)	Runs a SELECT and returns the rows as a list of dictionaries
rows_to_dicts(), json_value()	Convert result rows, DECIMAL and DATE values into JSON-friendly data
read_json(), error_response()	Read the request body safely; return {success: false, error: ...} with a status code
calculate_slabs(units)	Progressive slab calculation; returns the slab rows and the energy charge
get_consumers() / add_consumer(), get_connections() / add_connection(), get_usage() / add_usage()	List and insert consumers (with address, in one transaction), connections and meter readings; negative units and inactive connections are rejected
get_bills() / get_bill() / add_bill()	List bills, read one bill with its slabs, generate a bill transactionally
update_payment_status()	PATCH route that changes Payment_Status to Paid, Pending or Overdue
get_slab_breakdowns(), dashboard()	Slab rows of all bills; summary counts and totals for the dashboard
health(), not_found(), server_error()	Database health check; JSON error pages for unknown routes and server errors

Front-end functions (script.js): api() (Fetch wrapper), refresh() (loads all lists), render() (fills tables, counters and drop-downs), fillSelect(), formObject() (form to JSON), showBill() (printable bill with Print / Save as PDF) and money(), esc(), notify() (formatting, HTML escaping and status messages). Each form is sent with a POST request through api(), after which the page is refreshed.

4.7 Triggers

No database triggers are used in the present version of the project. The business rules are enforced in two other places: (i) the table definitions (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, ENUM and CHECK constraints and the ON DELETE rules shown in Section 4.1), and (ii) the Flask API, which validates the input, checks the connection status, prevents a second bill for the same reading and creates the bill and its slab rows in one transaction.

Two triggers that can be added later are proposed below. They were tested on a copy of the database: the first refuses a usage record for an inactive connection and the second prevents a paid bill from being deleted; both returned their error messages in the test.

```
DELIMITER //
CREATE TRIGGER trg_usage_active_only
BEFORE INSERT ON USAGE_RECORD
FOR EACH ROW
BEGIN
  DECLARE conn_status VARCHAR(10);
  SELECT Status INTO conn_status
  FROM CONNECTION_DETAILS
  WHERE Connection_ID = NEW.Connection_ID;
  IF conn_status <> 'Active' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot record usage for an inactive connection';
  END IF;
END//

CREATE TRIGGER trg_bill_keep_paid
BEFORE DELETE ON BILL
FOR EACH ROW
BEGIN
  IF OLD.Payment_Status = 'Paid' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'A paid bill cannot be deleted';
  END IF;
END//
DELIMITER ;
```

4.8 Sample Page

Fig 4.1 shows the home page of the application (VoltCare dashboard). The “MySQL connected” badge confirms that the Flask API can reach the database; the four counters (3 consumers, 5 active connections, 3 bills generated and 1 pending bill) are calculated from the data read from MySQL, and the cards below them open the modules used to register consumers and connections, record meter readings, generate bills and track payments.

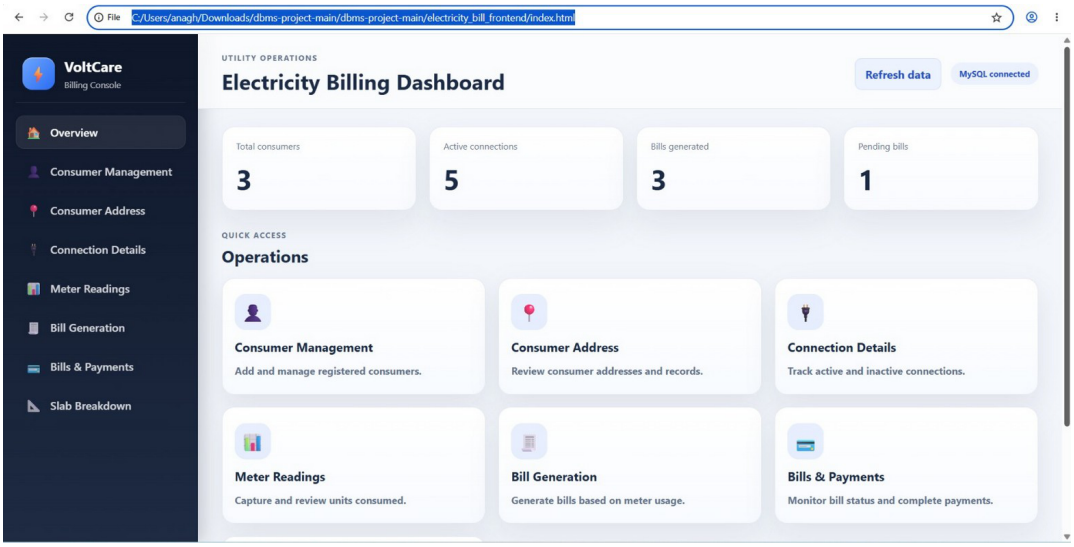


Fig 4.1: Home page of the VoltCare Electricity Bill Management System

5. SYSTEM TESTING

5.1 Test Cases

The test cases below were executed on the running system (MySQL 8.0 + Flask API + browser page). Each ‘Actual result’ is what the application returned when the test was run.

1. Consumer and Connection Module

Table 5.1: Test cases for consumers and connections

Test Case ID	Test scenario	Test steps / input	Expected result	Actual result	Status
TC01	Database connection	Open the page / call GET /api/health	Status badge shows “MySQL connected”	As expected	Pass
TC02	Register a consumer	Name, phone, email, address → Save consumer	Consumer saved; ID returned; shown in table	As expected	Pass
TC03	Missing fields	Send consumer without e-mail and address	Error: required fields	“Required fields: Email, Address”	Pass
TC04	Duplicate email	Register an existing e-mail	Rejected by UNIQUE constraint	Duplicate entry error (1062)	Pass
TC05	Add a connection	Select consumer, Domestic, new meter no.	Connection saved	As expected	Pass
TC06	Duplicate meter number	Add a connection with an existing Meter_No	Rejected by UNIQUE constraint	Duplicate entry error (1062)	Pass

2. Meter Reading Module

Table 5.2: Test cases for meter readings

Test Case ID	Test scenario	Test steps / input	Expected result	Actual result	Status
TC07	Add a reading	Active connection, today's date, 120 units	Reading saved	As expected	Pass
TC08	Negative units	Units = −5	Error message	"Units consumed must be a non-negative number."	Pass
TC09	Inactive connection	Reading for an Inactive connection	Reading refused	"Cannot record usage for an inactive connection."	Pass
TC10	Unknown connection	Connection_ID = 999	404 Not found	"Connection not found."	Pass

3. Bill Generation Module

Table 5.3: Test cases for bill generation

Test Case ID	Test scenario	Test steps / input	Expected result	Actual result	Status
TC11	Bill for 120 units, subsidy 20	Select unbilled record → Calculate & generate bill	Energy 475.00; final 455.00; 3 slab rows	475.00 / 455.00; 3 rows	Pass
TC12	Bill for 245 units	Generate bill, subsidy 0	Energy 1187.50; 4 slab rows	1187.50; 4 rows (162.50, 207.50, 525.00, 292.50)	Pass
TC13	Units inside first slab, subsidy larger than charge	45 units, subsidy 500	Energy 146.25; 1 slab row; final amount not negative	146.25; 1 row; final 0.00	Pass
TC14	Zero units	0 units, subsidy 0	Bill created with zero amount	Energy 0.00, no slab rows	Pass
TC15	Second bill for same reading	Generate bill again for the same record	Refused (1:1 relationship)	“A bill already exists for this usage record.”	Pass
TC16	Unknown usage record	Record_ID = 999	404 Not found	“Usage record not found.”	Pass
TC17	View bill details	GET /api/bills/{id}	Bill with its slab_breakdown list	As expected	Pass
TC18	Printable bill	After generating a bill in the page	Bill opens in a new tab with slab table	As expected	Pass
TC19	Slab total check	Run the integrity query of Section 4.3	Empty set (all slab totals equal Energy_Charge)	Empty set	Pass

4. Payment and General

Table 5.4: Test cases for payment status and general behaviour

Test Case ID	Test scenario	Test steps / input	Expected result	Actual result	Status
TC20	Mark a bill as paid	Click “Mark paid”	Status changes to Paid; button disappears	As expected (Fig 5.7)	Pass
TC21	Invalid payment status	PATCH with status “Done”	Request refused	“Payment_Status must be Paid, Pending, or Overdue.”	Pass
TC22	Payment for unknown bill	PATCH on bill 999	404 Not found	“Bill not found.”	Pass
TC23	Dashboard counters	GET /api/dashboard after the tests	Counts match the tables	As expected (Fig 4.1)	Pass
TC24	Unknown route	GET /api/nothing	JSON error, status 404	“API route not found.”	Pass

5.2 Screenshots of Work

The screenshots below were taken while running the project (VoltCare dashboard connected to MySQL). They show every module of the application with the demo data: three consumers (Anu Krishnan, Rahul Nair and Meera Das), their addresses and connections, the meter readings of 120, 245 and 45 units, the three generated bills and the automatic slab breakdown.

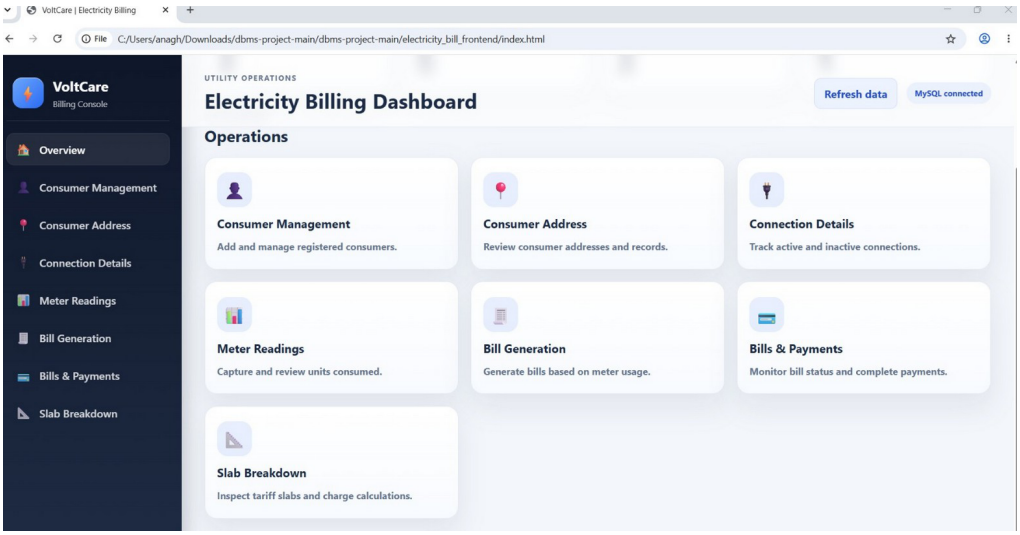


Fig 5.1: Operations menu of the dashboard (quick access to all modules)

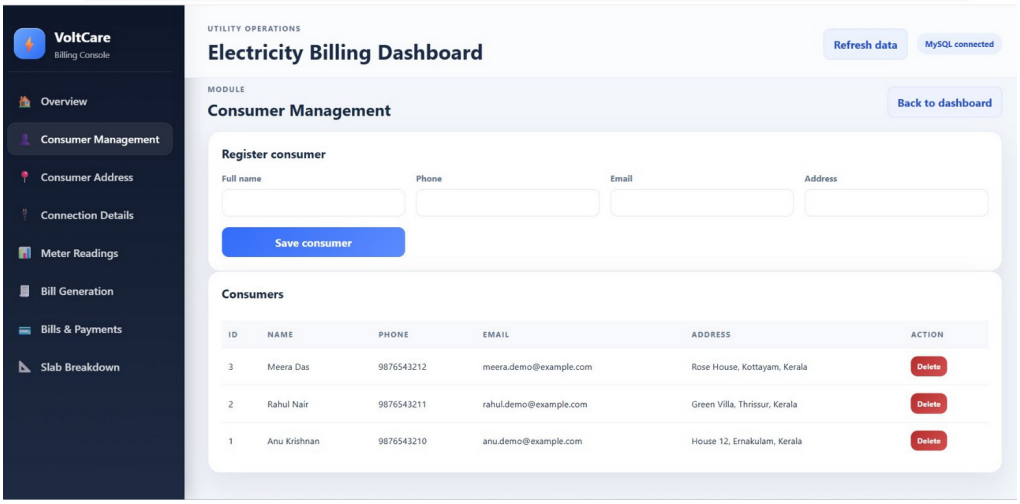


Fig 5.2: Consumer Management – register consumer form and list of consumers

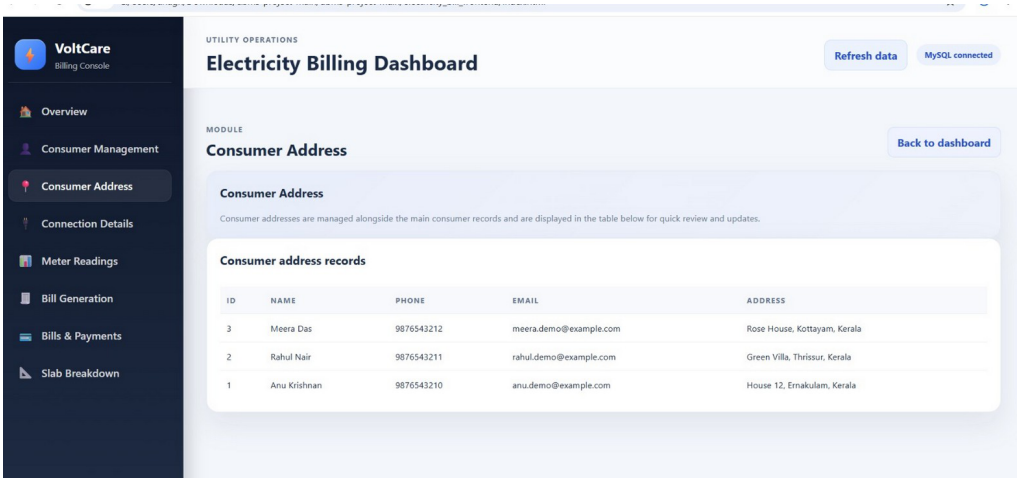


Fig 5.3: Consumer Address – addresses retrieved from CONSUMER_ADDRESS

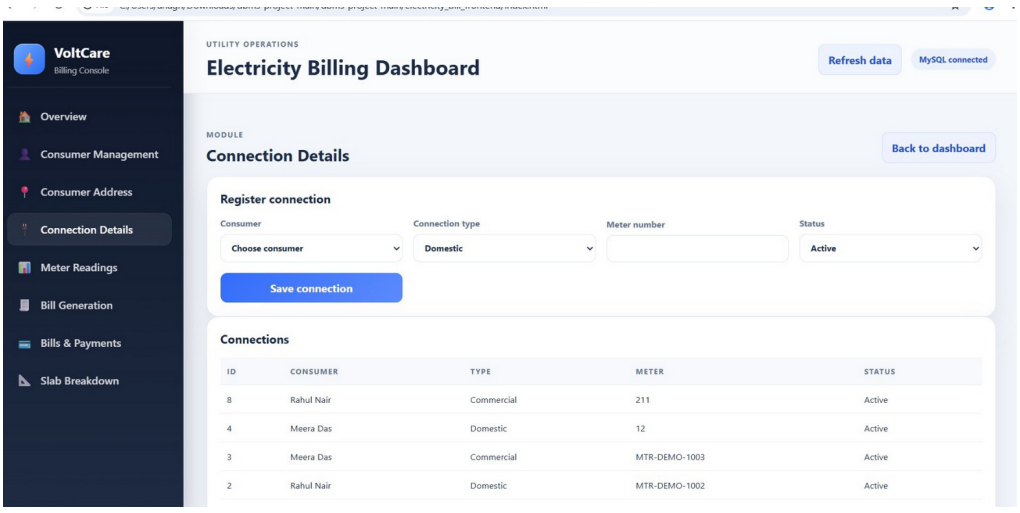


Fig 5.4: Connection Details – register connection form and list of connections with meter numbers and status

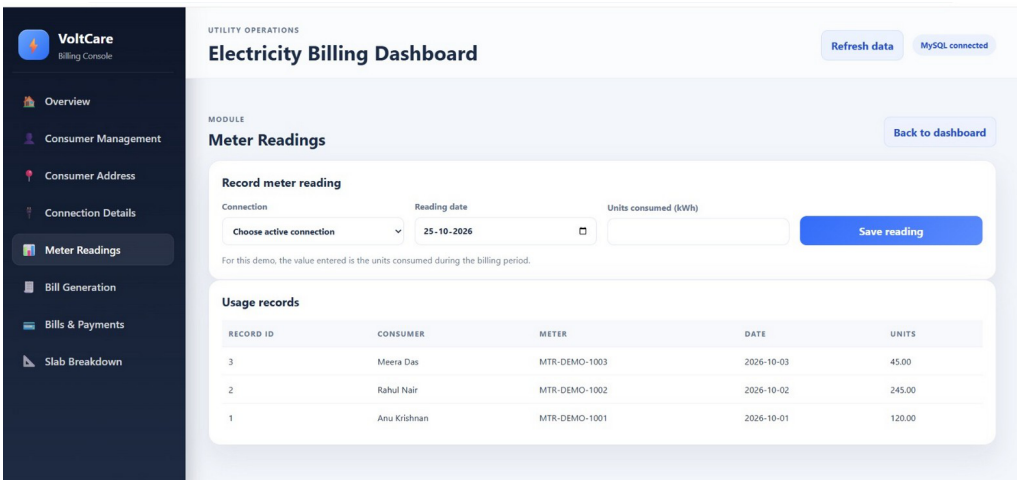


Fig 5.5: Meter Readings – record meter reading form and usage records (120, 245 and 45 units)

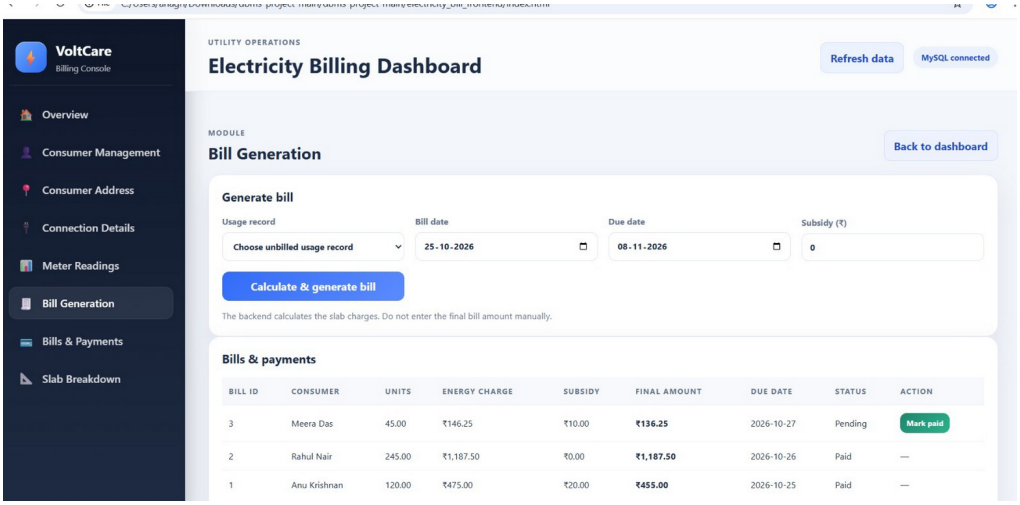


Fig 5.6: Bill Generation – the backend calculates the slab charges; the final amount is never typed manually

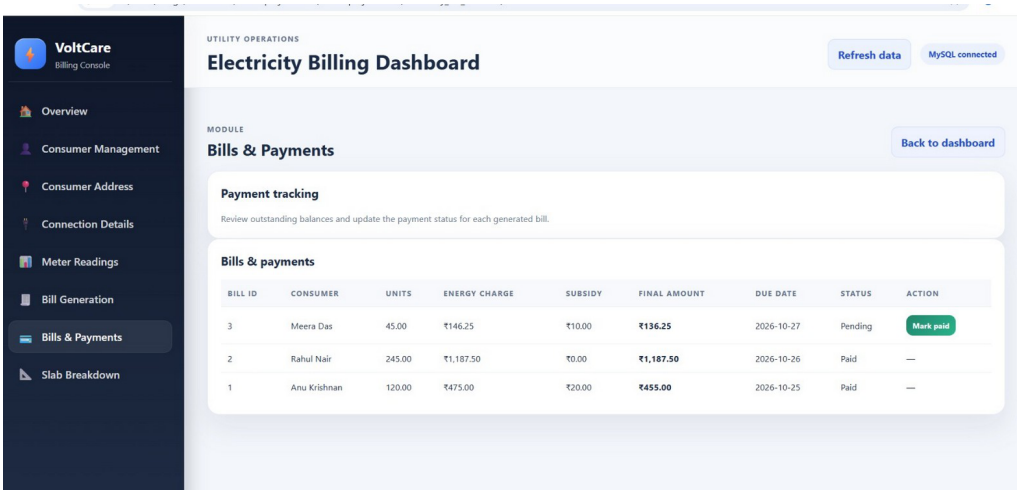


Fig 5.7: Bills & Payments – Bills 1 and 2 are Paid; Bill 3 (Rs. 136.25) is Pending with a "Mark paid" button

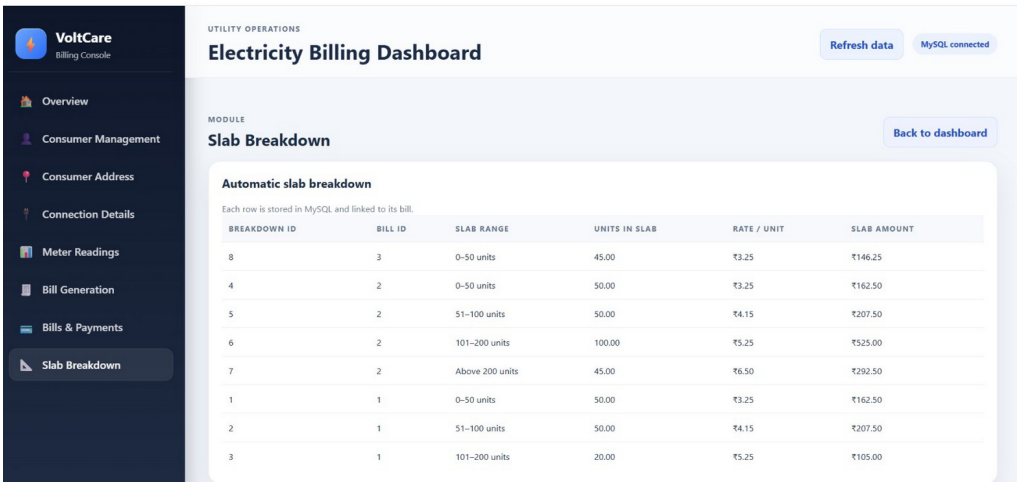


Fig 5.8: Slab Breakdown – automatic slab rows stored in MySQL and linked to each bill

6. CONCLUSION

The Electricity Bill Management System replaces manual billing with a reliable, database-driven process. The ER diagram was converted to a normalized relational schema of six tables in MySQL, and the whole billing cycle – registering a consumer, adding a connection, recording a meter reading, generating a bill and marking it paid – works end-to-end through a Flask REST API and a browser dashboard.

The central achievement of the project is the transparent slab-wise bill. The amount is never typed by a person: the back end calculates it with exact decimal arithmetic, stores the bill and every slab line in one transaction, and the consumer can see exactly how many units were charged at which rate. Constraints, unique keys, the one-bill-per-reading rule and the SQL integrity check keep the data consistent, and the test cases of Chapter 5 confirm that valid input is accepted and invalid input is refused with a clear message.

The normalization carried out in Section 3.3 showed that the schema satisfies 1NF, 2NF, 3NF and BCNF, so the data is stored without redundancy and without update, insertion or deletion anomalies. The project also gave the team practical experience of ER modelling, mapping to relations, normalization, writing DDL, DML and analytical SQL queries, connecting a database to an application, and testing a database-backed system. With the improvements listed in the next chapter, the system can grow into a complete billing solution.

7. FUTURE SCOPE

- **Authentication and roles:** separate logins for administrators, meter readers and consumers, so that a consumer can view only his or her own bills.
- **Late fine and overdue handling:** add a late-payment fine calculated from Due_Date and Payment_Status and change Pending bills to Overdue automatically (for example with a MySQL scheduled event).
- **Misuse detection:** implement the derived Flagged_Status to flag unusually high consumption, for example domestic connections used for commercial purposes.
- **Tariff table:** move the slab rates from the Python list into a database table with separate tariffs for Domestic, Commercial and Industrial connections and an effective-from date.
- **Meter-reading workflow:** store the previous and current meter reading and calculate Units_Consumed automatically.
- **Online payment and receipts:** integrate a payment gateway and record each payment in a PAYMENT table.
- **Notifications:** SMS or e-mail alerts for new bills, due dates and outages.
- **Edit and delete screens, reports and analytics:** add update/delete forms to the web page, month-wise revenue reports and charts.
- **Mobile application:** reuse the same REST API for an Android/iOS application.
- **Database triggers:** add the two triggers proposed in Section 4.7 to schema.sql so that the rules hold even when data is inserted from another tool.

8. APPENDIX

A. Project Folder Structure

```
electricity_billing_project/
  README.md
  electricity_bill_management_backend/
    app.py            Flask API and bill calculation
    schema.sql        database, tables and sample data
    requirements.txt  Flask, Flask-CORS, mysql-connector-python
  electricity_bill_frontend/
    index.html        dashboard page
    style.css         styling
    script.js         API calls and page logic
```

B. Implementation Steps

1. Install MySQL Server and run schema.sql in MySQL Workbench; this creates electricity_bill_db with its tables and sample data.
2. Open a terminal in the backend folder, create a virtual environment and run pip install -r requirements.txt.
3. Set DB_HOST, DB_USER, DB_PASSWORD and DB_NAME and start the API with py app.py (it listens on port 5000).
4. Open index.html from the frontend folder with VS Code Live Server; the badge should read “MySQL connected”.
5. Register a consumer and a connection, record a meter reading, generate the bill and mark it as paid.

9. REFERENCES

1. Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, Pearson.
2. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill.
3. MySQL 8.0 Reference Manual, Oracle Corporation (dev.mysql.com/doc).
4. Flask documentation, Pallets Projects (flask.palletsprojects.com).
5. MDN Web Docs: Fetch API (developer.mozilla.org).
6. Kerala State Electricity Board Ltd. (www.kseb.in) – for the slab-wise billing pattern.